

插槽Slots及具名插槽：实现Vue组件高度定制化

在Vue组件开发中，组件的复用性与定制化往往存在一定矛盾：过于通用的组件缺乏灵活性，而高度定制的组件又难以复用。**插槽（Slots）**正是解决这一矛盾的核心技术——它允许父组件向子组件“注入”自定义内容，让子组件在保持结构稳定性的同时，支持内容的灵活替换与扩展。本文将从基础插槽到具名插槽，完整解析其用法及定制化实践。

一、插槽的核心原理：组件通信的“内容通道”

插槽的本质是子组件中预留的“内容占位符”，父组件可以通过这个占位符传递任意HTML结构、其他组件甚至动态内容。其核心价值在于：

- 解耦结构与内容**：子组件负责定义固定结构（如卡片框架、表格布局），父组件负责填充个性化内容（如卡片标题、表格数据展示方式）；
- 提升组件复用性**：一套组件结构可适配不同业务场景的内容需求，无需重复开发相似组件；
- 支持灵活扩展**：无需修改子组件源码，即可通过父组件注入新内容实现功能扩展。

Vue中插槽的使用遵循“子组件留槽，父组件填内容”的原则，基础语法通过 `<slot></slot>` 标签实现。

二、基础插槽：最简单的内容注入

基础插槽（匿名插槽）是最基础的插槽形式，子组件中只定义一个无名称的插槽，父组件传递的所有内容会统一填充到这个插槽中。

1. 基础用法（Vue 3 组合式API）

步骤1：子组件定义插槽（预留占位符）

创建通用卡片组件 `CommonCard.vue`，在需要定制内容的区域插入 `<slot>` 标签，并可设置“默认内容”（当父组件未传递内容时显示）。

Code block

```
1 <template>
2   <div class="common-card">
3
4     <div class="card-header">
5
6       <slot>默认标题</slot>
7     </div>
```

```
8
9     <div class="card-body">
10
11         <slot>暂无内容</slot>
12     </div>
13
14     <div class="card-footer">
15         版权所有 © 2025
16     </div>
17 </div>
18 </template>
19
20 <script setup>
21 // 无需额外配置，仅通过模板中的<slot>标签定义插槽
22 </script>
23
24 <style scoped>
25 .common-card {
26     width: 300px;
27     border: 1px solid #eee;
28     border-radius: 8px;
29     padding: 16px;
30     margin: 10px;
31     box-shadow: 0 2px 4px rgba(0,0,0,0.1);
32 }
33 .card-header {
34     font-size: 18px;
35     font-weight: bold;
36     margin-bottom: 12px;
37 }
38 .card-footer {
39     margin-top: 12px;
40     font-size: 14px;
41     color: #666;
42     text-align: right;
43 }
44 </style>
```

步骤2：父组件填充内容（使用插槽）

在父组件 `App.vue` 中使用 `CommonCard`，并在组件标签内部写入需要注入的内容，这些内容会自动替换子组件中的 `<slot>` 标签（包括默认内容）。

Code block

```
1 <template>
```

```
2    <div class="app">
3
4      <CommonCard>
5        <template v-slot>
6          个人信息卡片
7        </template>
8      </CommonCard>
9
10
11     <CommonCard>
12       <div>
13         <span>商品详情</span>
14         <button style="margin-left: 10px;">收藏</button>
15       </div>
16       <p>商品名称: Vue实战教程</p>
17       <p>价格: 99元</p>
18     </CommonCard>
19
20
21     <CommonCard>
22       <UserAvatar />
23       <p>用户名: 前端开发工程师</p>
24     </CommonCard>
25   </div>
26 </template>
27
28 <script setup>
29 import CommonCard from './components/CommonCard.vue';
30 import UserAvatar from './components/UserAvatar.vue'; // 假设已定义头像组件
31 </script>
```

2. 基础插槽的局限性

上述示例中，子组件有两个 `<slot>` 标签（头部和主体），但父组件传递的内容会同时填充到两个插槽中，导致内容重复。这是因为**匿名插槽只能有一个**，无法区分不同的占位区域。此时，具名插槽应运而生。

三、具名插槽：区分不同占位区域的核心方案

具名插槽通过为 `<slot>` 标签设置 `name` 属性，明确区分不同的内容占位区域。父组件在传递内容时，通过指定目标插槽的名称，将内容精准注入到对应位置。

1. 具名插槽基础用法

步骤1：子组件定义具名插槽

修改 `CommonCard.vue`，为不同区域的插槽添加 `name` 属性，如“header”“body”，未命名的插槽默认名称为“default”。

Code block

```
1  <template>
2    <div class="common-card">
3
4      <div class="card-header">
5        <slot name="header">默认标题</slot>
6      </div>
7
8      <div class="card-body">
9        <slot name="body">暂无内容</slot>
10     </div>
11
12     <div class="card-footer">
13       <slot name="footer">版权所有 © 2025</slot>
14     </div>
15   </div>
16 </template>
17
18 <script setup></script>
19 <style scoped>/* 样式不变，省略 */</style>
```

步骤2：父组件精准注入内容

父组件通过 `<template v-slot:插槽名>`（简写为 `#插槽名`）指定内容要注入的目标插槽，实现“一对一”的内容匹配。

Code block

```
1  <template>
2    <div class="app">
3      <CommonCard>
4
5        <template #header>
6          <div>
7            商品详情卡片
8            <button style="margin-left: 10px;">收藏</button>
9          </div>
10        </template>
11
12      </div>
```

```
13     <template #body>
14       <div class="product-info">
15         <p>商品名称：Vue插槽实战指南</p>
16         <p>价格：59元</p>
17         <p>销量：1000+</p>
18       </div>
19     </template>
20
21
22     <template #footer>
23       <div>
24         <a href="/about">关于我们</a> |
25         <a href="/contact">联系客服</a>
26       </div>
27     </template>
28   </CommonCard>
29 </div>
30 </template>
31
32 <script setup>
33 import CommonCard from './components/CommonCard.vue';
34 </script>
```

此时，父组件传递的“header”“body”“footer”内容会精准对应子组件的三个插槽，实现了不同区域的独立定制，解决了匿名插槽的局限性。

2. 具名插槽的进阶用法：作用域插槽

在某些场景下，父组件注入插槽内容时，需要使用子组件内部的数据（如子组件列表中的某条数据）。此时，**作用域插槽**可以实现子组件向父组件的“数据传递”，让父组件在定制内容时能访问子组件的状态。

步骤1：子组件定义带数据的插槽

子组件通过 `<slot :数据名="数据值">` 的方式，将内部数据暴露给父组件，这些暴露的数据称为“插槽prop”。

Code block

```
1  <template>
2    <div class="user-list">
3      <h3>用户列表</h3>
4      <div v-for="user in userList" :key="user.id" class="user-item">
5
6        <slot name="user-item" :user="user" :index="$index">
7
```

```

8         {{ user.name }} - {{ user.age }}岁
9     </slot>
10 </div>
11 </div>
12 </template>
13
14 <script setup>
15 import { ref } from 'vue';
16
17 // 子组件内部数据
18 const userList = ref([
19   { id: 1, name: '张三', age: 25, gender: '男' },
20   { id: 2, name: '李四', age: 23, gender: '女' },
21   { id: 3, name: '王五', age: 28, gender: '男' }
22 ]);
23 </script>

```

步骤2：父组件接收并使用插槽prop

父组件通过 `#插槽名="插槽prop对象"` 的方式接收子组件暴露的数据，然后在内容中使用这些数据实现定制化展示。

Code block

```

1 <template>
2   <div class="app">
3     <UserList>
4
5       <template #user-item="slotProps">
6         <div class="custom-user">
7           <span>序号: {{ slotProps.index + 1 }}</span>
8           <span>姓名: {{ slotProps.user.name }}</span>
9           <span>年龄: {{ slotProps.user.age }}</span>
10          <span>性别: {{ slotProps.user.gender === '男' ? '♂' : '♀' }}</span>
11          <button @click="handleView(slotProps.user)">查看详情</button>
12        </div>
13      </template>
14    </UserList>
15  </div>
16 </template>
17
18 <script setup>
19 import UserList from './components/UserList.vue';
20
21 // 父组件方法，使用插槽传递的user数据
22 const handleView = (user) => {

```

```
23     console.log('查看用户详情: ', user);
24   };
25 </script>
```

也可通过**解构赋值**简化插槽prop的使用，让代码更简洁：

Code block

```
1   <template #user-item="{ user, index }">
2     <div class="custom-user">
3       <span>序号: {{ index + 1 }}</span>
4       <span>姓名: {{ user.name }}</span>
5
6     </div>
7   </template>
```

四、综合实战：用插槽实现高度定制化表格组件

表格是业务开发中最常用的组件之一，不同场景下表格的列展示、操作按钮差异很大。使用具名插槽+作用域插槽，可以实现一个通用表格组件，适配各种定制需求。

1. 通用表格组件（Table.vue）

Code block

```
1   <template>
2     <table class="common-table">
3
4       <thead>
5         <tr>
6           <slot name="header">
7
8             <th>默认列1</th>
9             <th>默认列2</th>
10          </slot>
11        </tr>
12      </thead>
13
14      <tbody>
15        <tr v-for="(row, index) in data" :key="row.id || index">
16          <slot name="body" :row="row" :index="index">
17
18            <td>{{ row.col1 }}</td>
19            <td>{{ row.col2 }}</td>
20          </slot>
```

```

21         </tr>
22     </tbody>
23
24     <tfoot>
25         <tr>
26             <slot name="footer"></slot>
27         </tr>
28     </tfoot>
29 </table>
30 </template>
31
32 <script setup>
33 import { defineProps } from 'vue';
34
35 // 接收父组件传递的表格数据
36 const props = defineProps({
37     data: {
38         type: Array,
39         required: true,
40         default: () => []
41     }
42 });
43 </script>
44
45 <style scoped>
46 .common-table {
47     width: 100%;
48     border-collapse: collapse;
49     margin: 20px 0;
50 }
51 .common-table th, .common-table td {
52     border: 1px solid #eee;
53     padding: 12px;
54     text-align: left;
55 }
56 .common-table th {
57     background-color: #f5f5f5;
58     font-weight: bold;
59 }
60 </style>

```

2. 父组件定制表格内容 (App.vue)

根据“用户管理”和“订单管理”两个场景，使用同一个 `Table` 组件，通过插槽定制不同的表头、表体和操作列。


```

1 <template>
2   <div class="app">
3
4     <h2>用户管理</h2>
5     <Table :data="userData">
6
7       <template #header>
8         <th>ID</th>
9         <th>姓名</th>
10        <th>年龄</th>
11        <th>操作</th>
12      </template>
13
14      <template #body="{ row }">
15        <td>{{ row.id }}</td>
16        <td>{{ row.name }}</td>
17        <td>{{ row.age }}</td>
18        <td>
19          <button @click="editUser(row)">编辑</button>
20          <button @click="deleteUser(row.id)" style="margin-left: 8px;">删除
21        </td>
22      </template>
23    </Table>
24
25
26    <h2>订单管理</h2>
27    <Table :data="orderData">
28
29      <template #header>
30        <th>订单号</th>
31        <th>商品名称</th>
32        <th>金额</th>
33        <th>状态</th>
34        <th>操作</th>
35      </template>
36
37      <template #body="{ row }">
38        <td>{{ row.orderNo }}</td>
39        <td>{{ row.goodsName }}</td>
40        <td>¥{{ row.amount.toFixed(2) }}</td>
41        <td>
42          <span :class="row.status === '已支付' ? 'paid' : 'unpaid'">{{
43            row.status }}</span>
44        </td>
45        <td>
46          <button @click="viewOrder(row)">查看订单</button>

```

```

46         </td>
47     </template>
48
49     <template #footer>
50         <td colspan="5" style="text-align: right;">
51             合计金额: ¥{{ totalAmount.toFixed(2) }}
52         </td>
53     </template>
54 </Table>
55 </div>
56 </template>
57
58 <script setup>
59 import { ref, computed } from 'vue';
60 import Table from './components/Table.vue';
61
62 // 用户数据
63 const userData = ref([
64     { id: 1, name: '张三', age: 25 },
65     { id: 2, name: '李四', age: 23 },
66     { id: 3, name: '王五', age: 28 }
67 ]);
68
69 // 订单数据
70 const orderData = ref([
71     { orderNo: 'OD2025001', goodsName: 'Vue实战教程', amount: 59, status: '已支付' },
72     { orderNo: 'OD2025002', goodsName: 'React入门指南', amount: 69, status: '未支付' },
73     { orderNo: 'OD2025003', goodsName: 'TypeScript手册', amount: 49, status: '已支付' }
74 ]);
75
76 // 计算订单合计金额
77 const totalAmount = computed(() => {
78     return orderData.value.reduce((sum, item) => sum + item.amount, 0);
79 });
80
81 // 操作方法
82 const editUser = (user) => console.log('编辑用户: ', user);
83 const deleteUser = (id) => console.log('删除用户ID: ', id);

```

五、插槽使用的核心注意事项

- 1. 插槽名大小写：**Vue 2中插槽名大小写不敏感（会自动转为小写），Vue 3中严格区分大小写，建议统一使用小写（如“header”而非“Header”）；

2. **v-slot的使用位置：** `v-slot` 只能用于 `<template>` 标签或组件标签本身，不能直接用于普通 HTML 标签；
3. **默认内容优先级：** 父组件传递了对应插槽内容时，子组件的默认内容会被覆盖，未传递则显示默认内容；
4. **作用域插槽数据隔离：** 父组件通过作用域插槽接收的子组件数据，仅在当前插槽内容中可用，不会污染父组件其他区域；
5. **Vue 2与Vue 3差异：** Vue 2中作用域插槽需用 `slot-scope` 属性，Vue 3中统一用 `v-slot`，迁移时需注意语法转换。

六、总结：插槽实现组件定制化的核心逻辑

插槽通过“子组件留槽、父组件填内容”的模式，完美解决了组件复用与定制化的矛盾。其中：

- **匿名插槽**适用于简单的内容注入场景，仅需一个定制区域；
- **具名插槽**通过名称区分不同占位区域，实现多区域独立定制；
- **作用域插槽**实现子组件数据向父组件的传递，让父组件定制内容时能关联子组件状态。

在实际开发中，合理运用插槽可以大幅提升组件的复用性和灵活性，尤其在开发通用组件（如表格、卡片、弹窗）时，插槽更是不可或缺的核心技术。

（注：文档部分内容可能由 AI 生成）